

StoreCraft Presentation

StoreCraft

Multi-tenant e-commerce platform on MySQL 8

COM2058 — Database Systems · Project Phase 3 Berk Kırık · Ankara University · Spring 2026

The problem

Independent merchants want their own online store — catalog, inventory, orders, payments — **without** spinning up a separate database each.

StoreCraft shares a single MySQL 8 schema across every tenant while enforcing **logical isolation** through `merchant_id` on every non-global relation.

17 entity types · 1 ternary relationship · 25 relationships · 22 base tables after mapping

Stack choices

Layer	Choice	Why
Database	MySQL 8.0 (InnoDB, utf8mb4)	Course target, strict SQL mode, generated columns
ORM	SQLAlchemy 2.0 (declarative)	Typed Mapped[] API, tenant filters stay explicit
Backend	FastAPI + Uvicorn	Async, auto-OpenAPI, dependency injection for sessions
Frontend	Jinja2 + HTMX	Server-rendered, zero-JS build toolchain
Seed	Faker (seed=42)	Deterministic, realistic Turkish + English locales
Orchestration	Docker Compose	docker compose up reproduces the full demo

ER → Relational mapping

Applied Elmasri & Navathe 6e · Chapter 9 rules:

Step	Rule	Applied to
1	Regular entity → relation	17 tables
2	Weak entity → compound PK	product_variants, order_items
3	1:1 partial/total → FK on partial side	merchants.owner_user_id
4	1:N → FK on N side	all HAS relationships
5	M:N → bridge table	merchant_staff, product_categories, cart_items, discount_usages
7	Ternary n-ary → bridge with n-part PK	inventory (STOCKED_AT)
8	IS-A specialization → per-subclass relation	customers, staff, platform_admins

Result: 17 + 5 bridges = **22 tables**.

Schema highlights

```
CREATE TABLE product_variants (  
  product_id BIGINT UNSIGNED NOT NULL,  
  variant_no INT UNSIGNED NOT NULL,  
  sku VARCHAR(80) NOT NULL,  
  option1_name VARCHAR(40) NOT NULL DEFAULT 'default',  
  ...  
  PRIMARY KEY (product_id, variant_no),      -- compound weak-  
    entity PK  
  UNIQUE KEY ux_pv_sku (sku),  
  CONSTRAINT fk_pv_product FOREIGN KEY (product_id)  
    REFERENCES products (product_id) ON DELETE CASCADE  
);  
  
CREATE TABLE inventory (  
  product_id BIGINT UNSIGNED NOT NULL,  
  variant_no INT UNSIGNED NOT NULL,  
  warehouse_id BIGINT UNSIGNED NOT NULL,  
  qty_on_hand INT NOT NULL DEFAULT 0,  
  qty_reserved INT NOT NULL DEFAULT 0,  
  PRIMARY KEY (product_id, variant_no, warehouse_id), -- ternary 3-  
    part PK  
  CONSTRAINT ck_inv_reserved_le_onhand
```

```
        CHECK (qty_reserved <= qty_on_hand)
    );
```

Integrity features used

- **Compound FK** for weak-entity references ((product_id, variant_no) → product_variants)
 - **Generated columns**: orders.grand_total = subtotal - discount + tax (STORED, indexed)
 - **CHECK constraints**: non-negative amounts, rating 1–5, commission 0–1, date windows
 - **ON DELETE CASCADE** for weak entities, **RESTRICT** for tenant references
 - **Unique indexes** on business keys (email, slug per tenant, SKU)
 - **Triggers**: loyalty points on paid, inventory reservation, discount usage bump
-

The seed data

python -m storecraft.scripts.seed populates **deterministic** Faker data:

Merchants

3

Users

~35

Products

48

Variants

~90

Orders

75–180

Payments

~120

Shipments

~80

Reviews

~30

3 merchants: **Berk’in Kitapçısı** (TR, books), **Ankara Elektronik** (TR, electronics), **TechStore** (US, gadgets).

Query showcase — the 25

Numbered Q1–Q25 in sql/999_showcase_queries.sql, mirrored as Python in queries/showcase.py.

Band	Queries	Operations
Basics	Q1–Q5	SELECT, WHERE, ORDER, LIMIT, DISTINCT, string/date functions
JOINS	Q6–Q10	INNER, LEFT, 4-way, SELF, anti-join via LEFT + IS NULL
Subqueries	Q11–Q13	scalar, IN, correlated EXISTS
Aggregates	Q14–Q17	GROUP BY, HAVING, conditional SUM, ROLLUP
Window	Q18–Q20	ROW_NUMBER, LAG + running SUM, NTILE
CTE	Q21–Q22	WITH, recursive CTE for category tree
DML + txn	Q23–Q25	VIEW usage, UPDATE with JOIN, BEGIN/INSERT/ROLLBACK

Example: recursive CTE (Q22)

```

WITH RECURSIVE category_tree (category_id, merchant_id, name,
    parent_category_id, depth, path) AS (
    SELECT category_id, merchant_id, name, parent_category_id, 0, name
    FROM categories
    WHERE parent_category_id IS NULL AND merchant_id = :mid
    UNION ALL
    SELECT c.category_id, c.merchant_id, c.name, c.parent_category_id,
        t.depth + 1, CONCAT(t.path, ' > ', c.name)
    FROM categories c
    JOIN category_tree t ON c.parent_category_id = t.category_id
)
SELECT merchant_id, depth, path, category_id
FROM category_tree
ORDER BY merchant_id, path;

```

Traverses CATEGORIES → PARENT_OF → CATEGORIES (R9) to arbitrary depth.

Example: window function (Q18)

```

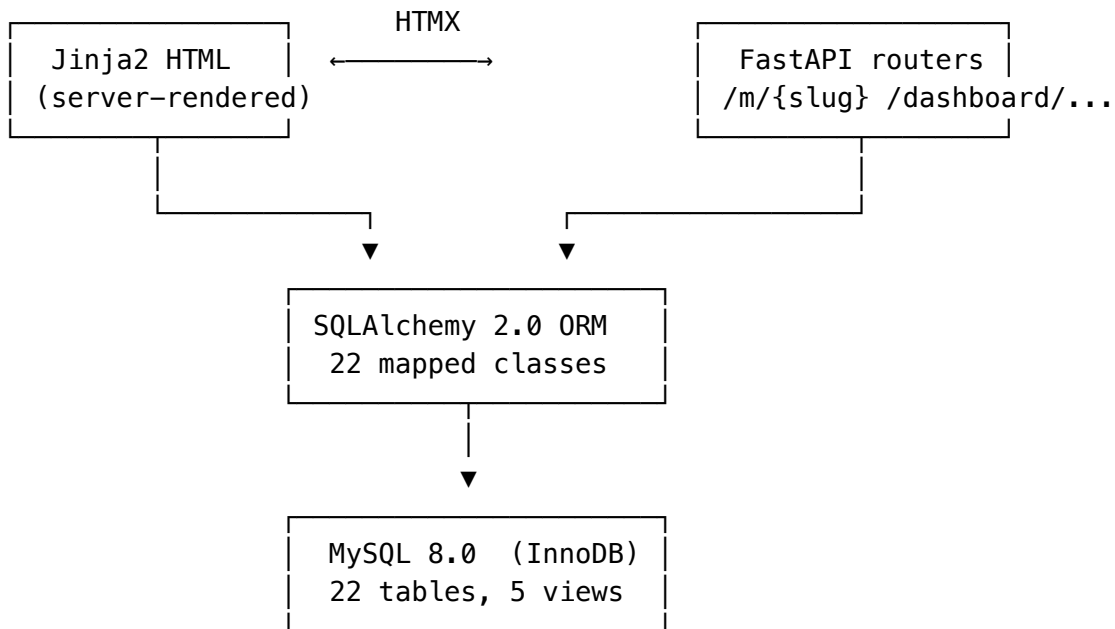
SELECT merchant_id, customer_name, lifetime_spend,
    ROW_NUMBER() OVER (
        PARTITION BY merchant_id
        ORDER BY lifetime_spend DESC
    ) AS rank_in_store
FROM v_customer_lifetime_value

```

```
WHERE lifetime_spend > 0  
ORDER BY merchant_id, rank_in_store;
```

Ranks every customer inside each merchant scope. Drives the **Top customers** table on the merchant dashboard.

Application architecture



Demo 1 — public storefront

<http://localhost:8000/m/berkin-kitapcisi>

- Category sidebar (seeded from R10 bridge)
 - Product grid with HTMX search (hx-get on input delay:300ms, swaps #products partial)
 - Product detail: variant table, reviews aggregate, rating stars
-

Demo 2 — merchant dashboard

<http://localhost:8000/dashboard/berkin-kitapcisi>

- KPI strip: orders count, gross sales, AOV, this-month revenue (**Q14 + Q21**)
- Top products table (**view v_top_products_by_merchant**)
- Low-stock alerts (**view v_low_stock_alerts**)
- Customer rank (**Q18 window function**)

Every widget runs a real SQL query — no precomputation.

Demo 3 — platform admin

`http://localhost:8000/admin`

- Merchant directory with plan badges and status
 - Last-7-day activity log (**view v_recent_activity**, resolves `actor_user_id` to name)
 - Polymorphic association in action: `entity_type:entity_id` reference without FK
-

Testing

`docker compose exec app pytest`

20 passed, 5 skipped in 0.18s

- **Model tests** — unique email, cascade delete for weak entities, compound PK integrity, ternary inventory PK, recursive FK
 - **API tests** — route smoke tests, HTMX header distinguishes partial vs full, 404 handling
 - **Tenant isolation** — same slug allowed across tenants, `merchant_id` filter enforced
 - **MySQL-only queries** — 5 skipped under SQLite, verified via docker compose
-

Lessons learned

1. **Weak entities map cleanly** — compound FKs everywhere, ORM exposes them via `ForeignKeyConstraint`
 2. **Ternary as a full table** is simpler than nested bridges; recursive CTE later proved it was worth it
 3. **Computed columns** replaced a whole class of app-layer bugs (`grand_total` arithmetic)
 4. **HTMX over SPA** — one codebase, zero bundler, same-origin cookies work out of the box
 5. **Thick-line / thin-line** (instead of double-line) on drawio was the cleanest convention for total vs partial participation in Chen notation
-

What's next (out of scope for Phase 3)

- **RBAC** via `merchant_staff.role` — currently enum only
- **Returns / refunds workflow** — status field exists, trigger cascade not yet wired
- **i18n** — prices display in merchant currency; user-facing text is English
- **Event sourcing** — `activity_log` is the foundation; projections could feed realtime dashboards

Phase 4 report has full details of each.

Thank you

Berk Kırık · Ankara University Computer Engineering · COM2058 Spring 2026

- Repo: github.com/Berkkirik/com2058
- Phase 1: docs/phase1_data_requirements.md
- Phase 2: docs/phase2_er_diagram.drawio
- Phase 3: phase3/
- Phase 4 report: docs/phase4_report.pdf